

DCT2101 – Sistemas Operacionais

Sincronização de Processos

João Borges

Universidade Federal do Rio Grande do Norte – UFRN
Centro de Ensino Superior do Seridó – CERES
Departamento de Computação e Tecnologia – DCT

Agenda

- 1 Introdução
- 2 Seção crítica
- 3 Problemas Clássicos
- 4 Estudo de caso

Introdução

- Programa executado por apenas um processo é dito de programa sequencial
 - Existe apenas um fluxo de controle
- Programa **concorrente** é executado por diversos processos que cooperam entre si para realização de uma tarefa (aplicação)
 - Existem vários fluxos de controle
 - Necessidade de interação para troca de informações (sincronização)
- Emprego de termos
 - Paralelismo real: só ocorre em máquinas multiprocessadoras
 - Paralelismo “aparente” (concorrência): máquinas monoprocessadoras
 - Execução simultânea versus estar “em estado de execução” simultaneamente

Programação concorrente

- Composta por um conjunto de processos sequenciais que executam concorrentemente
- Processos disputam recursos comuns
 - variáveis, periféricos, etc...
- Um processo é dito de cooperante quando é capaz de afetar, ou ser afetado, pela execução de outro processo
- Exemplo: Livro SD, *concorrentes.c*, Cap 4, p. 105.

Vantagens e desvantagens da prog. concorrente

Vantagens

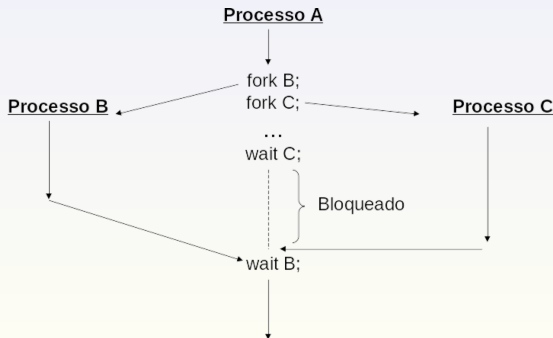
- Aumento de desempenho:
 - Permite a exploração do paralelismo real disponível em máquinas multiprocessadoras
 - Sobreposição de operações de E/S com processamento
- Facilidade de desenvolvimento de aplicações que possuem um paralelismo intrínseco

Desvantagens

- Programação complexa
- Aos erros “comuns” se adicionam erros próprios ao modelo
 - Diferenças de velocidade relativas de execução dos processos
- Aspecto não determinístico
 - Difícil depuração

Aspectos da programação concorrente

- Processos paralelos podem ser executados em qualquer ordem
 - Duas execuções consecutivas do mesmo programa, com os mesmos dados de entrada, podem gerar resultados diferentes
 - Não é necessariamente um erro
 - Possibilidade de forçar a execução em uma determinada ordem

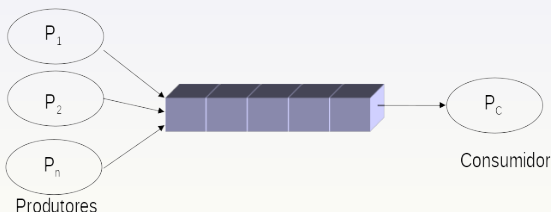


O problema do compartilhamento de recursos

- A programação concorrente implica em um compartilhamento de recursos
 - Variáveis compartilhadas são recursos essenciais para a programação concorrente
- Acesso concorrente a dados compartilhados pode resultar em inconsistências.
- Acessos a recursos compartilhados devem ser feitos de forma a manter um estado coerente e correto do sistema.
- Manter a consistência de dados requer a utilização de mecanismos para garantir a execução ordenada de processos cooperantes.
- Exemplo : Livro SD, *exclusaomutua.c*, Cap 5, p. 114.

Exemplo: relação produtor-consumidor

- Relação produtor-consumidor é uma situação bastante comum em sistemas operacionais
- Exemplo: Servidor de impressão
 - Processos usuários produzem “impressões”
 - Impressões são organizadas em uma fila a partir da qual um processo (consumidor) os lê e envia para a impressora



Exemplo: relação produtor-consumidor

- Suposições para implementação:
 - Suponha que seja desejado fornecer uma solução para o problema do produtor-consumidor que utilize todo o buffer.
 - É possível fazer isso tendo um inteiro **count** que mantém o número de posições ocupadas no buffer.
 - **count** é uma variável compartilhada.
 - Inicialmente, **count** é inicializado em 0.
 - Ele é incrementado pelo produtor após a produção de um novo item.
 - E decrementado pelo consumidor após a retirada.

Exemplo: relação produtor-consumidor

Código do Produtor

```
while (true)
{
    /* produz um item e coloca em nextProduced */
    while (count == BUFFER_SIZE)
        ; // do nothing

    buffer [in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    count++;
}
```

Exemplo: relação produtor-consumidor

Código do Consumidor

```
while (true)
{
    while (count == 0)
        ; // não faz nada

    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    count--;

    /* consome o item em nextConsumed */
}
```

Detalhes Produtor-Consumidor

- `count++` pode ser implementado como

```
register1 = count
register1 = register1 + 1
count = register1
```

- `count--` pode ser implementado como

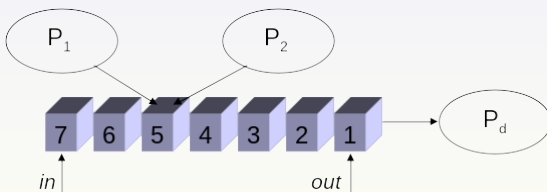
```
register2 = count
register2 = register2 - 1
count = register2
```

- Considere a seguinte ordem de execução, com `count = 5`:

- 1 Produtor: `register1 = count` // `register1 = 5`
- 2 Produtor: `register1 = register1 + 1` // `register1 = 6`
- 3 Consumidor: `register2 = count` // `register2 = 5`
- 4 Consumidor: `register2 = register2 - 1` // `register2 = 4`
- 5 Produtor: `count = register1` // `count = 6`
- 6 Consumidor: `count = register2` // `count = 4`

Exemplo: relação produtor-consumidor

- Sequência de operações:
 - 1 P1 vai imprimir; lê valor de *in* (5); perde processador
 - 2 P2 ganha processador; lê valor de *in* (5); insere arquivo; atualiza *in* (6)
 - 3 P1 ganha processador; insere arquivo (5); atualiza *in* (7)
- Estado incorreto:
 - Impressão de P2 é perdida
 - Na posição 6 não há uma solicitação válida de impressão



O problema da seção crítica

- Condição de corrida (*race condition*):
 - Situação que ocorre quando vários processos manipulam o mesmo conjunto de dados concorrentemente,
 - E o resultado depende da ordem em que os acessos são feitos.
- Seção (região) crítica:
 - Segmento de código no qual um processo realiza a alteração de um recurso compartilhado.
- Preempção:
 - Kernel com preempção: permite que um processo seja interceptado enquanto está sendo executado
 - Kernel sem preempção: não permite que um processo em execução seja interceptado, será executado até concluir ou ser bloqueado.

Necessidade da programação concorrente

- Eliminar corridas
- Criação de um protocolo para permitir que processos possam cooperar sem afetar a consistência dos dados
- Controle de acesso à seção crítica:
 - Garantir a exclusão mútua

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```


Solução para o problema da seção crítica

- Regra 1 - Exclusão mútua
 - Dois ou mais processos não podem estar simultaneamente em uma seção crítica.
 - Se um processo P_i está executando sua seção crítica, então nenhuma seção crítica de outro processo pode estar sendo executada.
- Regra 2 - Progresso
 - Nenhum processo fora da seção crítica pode bloquear a execução de um outro processo.
 - Se nenhum processo está executando uma seção crítica e existem processos que desejam entrar nas seções críticas deles, então a escolha do próximo processo que irá entrar na seção crítica não pode ser adiada indefinidamente.

Solução para o problema da seção crítica

- Regra 3 - Espera limitada

- Nenhum processo deve esperar infinitamente para entrar em uma seção crítica.
 - Existe um limite para o número de vezes que outros processos são selecionados para entrar nas seções críticas deles, depois que um processo fez uma requisição para entrar em sua seção e antes que essa requisição seja atendida.

- Regra 4

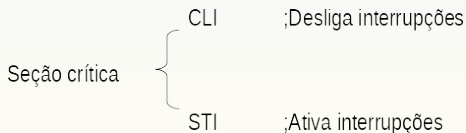
- Não fazer considerações sobre o número de processadores, nem de suas velocidades relativas.
 - É assumido que cada processo executa em uma velocidade diferente de zero
 - Nenhuma hipótese é feita referente à velocidade relativa de execução dos N processos.

Obtenção da exclusão mútua

- Desabilitando interrupções
- Variáveis especiais do tipo lock
- Alternância de execução
 - Inicialmente será apresentada uma tentativa e depois a solução

Desabilitando interrupções

- Muitos sistemas fornecem suporte de hardware para código de seção crítica
- Sistemas Monoprocessados - podem desabilitar interrupções
 - Código em execução pode executar sem preempção
 - Não há troca de processos com a ocorrência de interrupções de tempo ou de eventos externos
- Desvantagens:
 - Poder demais para um usuário
 - Não funciona em máquinas multiprocessadoras (SMP) pois apenas a CPU que realiza a instrução é afetada (violação da Regra 4).



Variáveis do tipo lock

- Criação de uma variável especial compartilhada que armazena dois estados:
 - 0 - livre
 - 1 - ocupado
- Desvantagem:
 - Apresenta *condições de corrida*

Seção crítica {
While (lock==1);
lock=1;

lock=0;

Alternância: Tentativa

- Variáveis compartilhadas:

- var **turn**: (0..1);
- Inicialmente: turn = 0
- Se turn = i
 - P_i pode entrar na sua região crítica

- Processo P_i

```
do {
    while turn != i ; /* não faz nada */
    // SEÇÃO CRÍTICA
    turn = j;
    // SEÇÃO RESTANTE
} while (TRUE);
```

- Satisfaz exclusão mútua, mas não progresso.

- Viola a regra 2 se a parte não crítica de um processo for muito maior que a do outro

Alternância: Solução de Peterson

- Os dois processos compartilham duas variáveis:
 - `int turn;`
 - `boolean flag[2];`
- A variável **turn** indica de quem é a vez de entrar na região crítica.
- O vetor **flag** é usado para indicar se um processo está pronto para entrar na região crítica..
 - `flag[i] = true` significa que processo P_i está pronto!
- Solução algorítmica:
 - Combinação de variáveis do tipo lock e alternância (Dekker 1965, Peterson 1981)
 - Exemplo : Livro SD, exclusaomutua2.c, Cap 5, p. 116-117.

Alternância: Solução de Peterson

- Algoritmo para processo P_i

```
do {  
    flag[i] = TRUE;  
    turn = j;  
    while ( flag[j] && turn == j); /* não faz nada */  
  
    // SEÇÃO CRÍTICA  
  
    flag[i] = FALSE;  
  
    // SEÇÃO RESTANTE  
  
} while (TRUE);
```


Análise da solução de Peterson

- Desvantagem:
 - Teste contínuo do valor da variável compartilhada provoca o desperdício do tempo do processador (*busy waiting*).
- Solução:
 - Bloquear o processo ao invés de executar *busy waiting*
 - Baseado em duas novas primitivas:
 - sleep: Bloqueia um processo a espera de uma sinalização
 - wakeup: Sinaliza um processo

Semáforos

- Mecanismo proposto por Dijkstra (1965)
- Ferramenta de sincronização que não requer espera ocupada (*busy waiting*)
- Duas operações padrão:
 - `wait()`
 - Originalmente chamada `P()` (proberen, testar)
 - `signal()`
 - Originalmente `V()` (verhogen, incrementar)
- Possui duas versões:
 - Menos complicada
 - Mais complexa

Semáforos: versão mais simples

- O semáforo S é uma variável inteira
- Somente pode ser acessada via duas operações indivisíveis (atômicas):

```
wait (S)
```

```
{
```

```
    while S <= 0 ; // não faz nada
```

```
    S--;
```

```
}
```

```
signal (S)
```

```
{
```

```
    S++;
```

```
}
```

Semáforo como uma Ferramenta Geral de Sincronização

- Semáforo **Contador** - valor nele armazenado pode ser qualquer número inteiro.
- Semáforo **Binário** - valor inteiro nele armazenado pode variar entre 0 e 1; pode ser implementado mais simplesmente.
 - Também conhecido como *mutex locks*
- Fornece exclusão mútua:

```
Semaphore mutex;    // inicializado em 1
do
{
    wait (mutex);
        // Seção Crítica
    signal (mutex);
        // restante do código
} while (TRUE);
```

Semáforo como uma Ferramenta Geral de Sincronização

- Utilizando um semáforo binário para sincronizar a ordem de execução de dois processos:

- 1 P_1 executará instrução S_1
- 2 P_2 executará instrução S_2

- Iniciando um semáforo compartilhado (synch) como 0:

```
Semaphore synch;    // inicializado em 0
```

```
// P1
```

```
S1;
```

```
signal (synch);
```

```
// P2
```

```
wait(synch);
```

```
S2;
```

Implementação de Semáforo mais simples

- Deve garantir que dois processos não possam executar `wait()` e `signal()` no mesmo semáforo ao mesmo tempo.
- Daí, a implementação se torna o problema da seção crítica na qual o código do `wait` e `signal` são colocados em seções críticas.
 - Pode ter espera ocupada na implementação da seção crítica
 - Código de implementação é menor
 - Pequena espera ocupada se seção crítica está sendo usada raramente
- Observe que aplicações podem perder muito tempo em seções críticas e daí esta não é uma boa solução.

Semáforos: Implementação sem Espera Ocupada

- O semáforo é um tipo abstrato de dados:
 - Um valor inteiro
 - Fila de processo
- Associar uma fila de espera com cada semáforo.
- Cada entrada na fila de espera tem dois itens:
 - valor (de tipo inteiro)
 - ponteiro para o próximo registro na lista.
- Duas operações:
 - **block** - coloca o processo que evoca a operação na fila de espera apropriada.
 - **wakeup** - remove um processo da fila de espera e coloca-o na fila de processos aptos/prontos (*ready queue*).

Semáforos: Implementação sem Espera Ocupada

- Implementação de wait:

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        adiciona esse processo em S->list;  
        block();  
    }  
}
```

- Implementação de signal:

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove o processo P de S->list;  
        wakeup(P);  
    }  
}
```


Deadlock (Impasse) e Starvation (Abandono)

- **Deadlock** - dois ou mais processos estão esperando indefinidamente por um evento que pode ser causado somente por um dos processos esperando o evento

- Seja S e Q dois semáforos inicializados em 1

// P_0

wait (S);

wait (Q);

....

signal (S);

signal (Q);

// P_1

wait (Q);

wait (S);

...

signal (Q);

signal (S);

- **Starvation** - bloqueio indefinido. Um processo pode nunca ser removido da fila do semáforo em que está suspensa
- **Priority Inversion** - inversão de prioridade. Problema de escalonamento em que um processo de baixa prioridade mantém um *lock* necessário para um processo de maior prioridade

Problemas Clássicos de Sincronização

- Problema do Buffer de tamanho limitado (Bounded-Buffer)
- Problema dos Leitores e Escritores
- Problema do Jantar dos Filósofos (Dining-Philosophers)

Problema do Buffer de Tamanho Limitado

- N posições, cada um pode armazenar um item
- Semáforo **mutex** inicializado com o valor 1
- Semáforo **full** inicializado com o valor 0
- Semáforo **empty** inicializado com o valor N .

Problema do Buffer de Tamanho Limitado

- A estrutura do processo produtor

```
do
```

```
{
```

```
    // produz um item
```

```
    wait (empty);
```

```
    wait (mutex);
```

```
    // adiciona o item ao buffer
```

```
    signal (mutex);
```

```
    signal (full);
```

```
} while (TRUE)
```

Problema do Buffer de Tamanho Limitado

- A estrutura do processo consumidor

```
do
```

```
{
```

```
    wait (full);
```

```
    wait (mutex);
```

```
        // remove um item do buffer
```

```
    signal (mutex);
```

```
    signal (empty);
```

```
        // consome o item removido
```

```
} while (TRUE);
```

Problema dos Leitores e Escritores

- Um conjunto de dados é compartilhado entre vários processos concorrentes
 - **Leitores** - somente lê um conjunto de dados; eles não realizam nenhuma atualização
 - **Escritores** - podem ler e escrever.
- Problema:
 - Permitir múltiplos leitores ler ao mesmo tempo.
 - Somente um único escritor pode acessar os dados compartilhados ao mesmo tempo.
- Dados Compartilhados
 - Conjunto de dados
 - Semáforo **mutex** inicializado em 1.
 - Semáforo **wrt** inicializado em 1.
 - Inteiro **readcount** inicializado em 0.

Problema dos Leitores e Escritores

- A estrutura de um processo escritor

```
do
{
    wait (wrt) ;

    //      writing is performed

    signal (wrt) ;
} while (TRUE);
```

Problema dos Leitores e Escritores

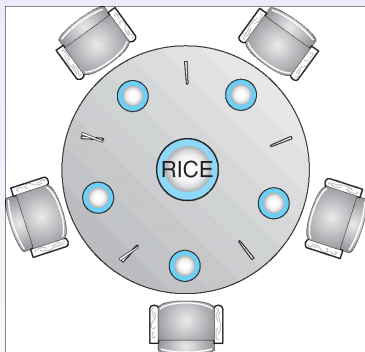
- A estrutura de um processo leitor

```
do
{
    wait (mutex) ;
    readcount++ ;
    if (readcount == 1)
        wait (wrt);
    signal (mutex)

    // reading is performed

    wait (mutex);
    readcount--;
    if (readcount == 0)
        signal (wrt);
    signal (mutex) ;
} while (TRUE);
```


Problema do Jantar dos Filósofos



- Dados Compartilhados
 - Tigela de Arroz (conjunto de dados)
 - Semáforo chopstick [5] inicializados em 1

Problema do Jantar dos Filósofos

- A estrutura do Filósofo i :

```
do
```

```
{
```

```
    wait ( chopstick[i] );
```

```
    wait ( chopstick[ (i + 1) % 5] );
```

```
        //  comer
```

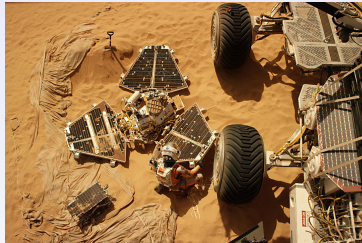
```
    signal ( chopstick[i] );
```

```
    signal ( chopstick[ (i + 1) % 5] );
```

```
        //  pensar
```

```
} while (TRUE);
```

A martian bug¹: O problema



- Missão Mars Pathfinder (1997) rodava um RTOS da VxWorks.
- Após pouso, o sistema da Pathfinder reiniciava constantemente, atrasando a transmissão dos dados coletados pra terra.
- Foram 3 semanas para descobrir o problema e 18 horas para resolver.

¹Ars Technica. Jacek Krywko. 2/Oct/2020.

<https://arstechnica.com/features/2020/10/>

[the-space-operating-systems-booting-up-where-no-one-has-gone-before/](https://arstechnica.com/features/2020/10/the-space-operating-systems-booting-up-where-no-one-has-gone-before/)

A martian bug: O motivo

- Espaço de memória **compartilhado** (Information bus) usado para troca de dados entre diferentes componentes (processos).
- Esse espaço era controlado com um **semáforo mutex**.
- Três tarefas envolvidas nos reboots misteriosos:
 - ① *Alta prioridade*: tarefa para gerenciar a operação do *information bus*.
 - ② *Baixa prioridade*: de vez em quando pegava o information bus para compartilhar dados meteorológicos.
 - ③ *Média prioridade*: tarefa de comunicação.

A martian bug: Como deveria funcionar

- A tarefa meteorológica (2) deveria pegar o information bus muito raramente.
- Em raras ocasiões:
 - A tarefa de gerenciamento (1) era escalonada para rodar.
 - Enquanto a tarefa (2) estava rodando.
 - A tarefa (1) de alta prioridade tentava pegar o mesmo **mutex**.
 - Ficando bloqueada até que a tarefa (2) de baixa prioridade utilizasse o bus e liberasse o recurso.
- Isso é o ideal, pois a transferência dos dados da tarefa (2) deveria iniciar e finalizar corretamente.
- Mas, a tarefa de comunicação (3) de prioridade média entrava em cena e causava o problema.

A martian bug: Como ocorria o problema

- Uma sequência muito improvável de eventos:
 - A tarefa (3) de média prioridade era escalonada pra rodar.
 - Enquanto a tarefa (2) estava rodando,
 - E tinha causado o boqueio da tarefa (1) no mutex.
- Havia uma janela de tempo muito pequena para isso acontecer, mas quando acontecia:
 - A tarefa (3) preemptava a tarefa (2) de baixa prioridade.
 - Com isso, a tarefa (2) não liberava o mutex para a tarefa (1) de alta prioridade.
- Consequentemente, a tarefa (3), indiretamente, bloqueava a tarefa (1): **inversão de prioridades**.
 - A tarefa (1) entrava em estado bloqueado.
 - Quando uma outra tarefa independente do kernel percebia que uma tarefa importante não estava rodando como planejado, ele iniciava um reboot total.

A martian bug: A solução

- Os reboots aconteceram algumas vezes em 2 semanas.
- Mas o sistema já possuía uma solução para isso:
 - **Herança de prioridades.**
 - Mas que havia sido desativada (erro humano).
- Como funcionava:
 - A tarefa (2) de baixa prioridade, herdava a prioridade da tarefa (1) de maior prioridade que estava esperando pelo semáforo mutex.
 - Isso impedia a tarefa de média prioridade (3) de preemptá-la.